SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr. Hemavathi R

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	**5**
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	**5**
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	**5**
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	**5**
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	**5**
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	**5**
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	**5**

8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	**5**
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	**5**
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	**5**

Code of Conduct:

I Sree B(192524023) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Sree B

1.Given the scenario of a School Management System, identify relations and write SQL to list all students

with their ass teacher and grade

```
mysql> CREATE TABLE Class (
    ->     class_id INT PRIMARY KEY,
    ->     class_name VARCHAR(20),
    ->     teacher_id INT,
    ->     FOREIGN KEY (teacher_id) REFERENCES Teacher(teacher_id)
    -> );
Query OK, 0 rows affected (0.545 sec)

mysql>
mysql> -- Student Table
Query OK, 0 rows affected (0.005 sec)

mysql> CREATE TABLE Student (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(50),
    ->     class_id INT,
    ->     grade CHAR(2),
    ->     FOREIGN KEY (class_id) REFERENCES Class(class_id)
    -> );
Query OK, 0 rows affected (0.979 sec)

mysql> INSERT INTO Teacher VALUES
    -> (1, 'Mr. Kumar'),
    -> (2, 'Ms. Priya');
Query OK, 2 rows affected (0.556 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Class VALUES
    -> (101, 'Class 10A', 1),
    -> (102, 'Class 10B', 2);
Query OK, 2 rows affected (0.070 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student VALUES
    -> (1001, 'Rahul', 101, 'A'),
    -> (1002, 'Sneha', 101, 'B'),
    -> (1003, 'Arjun', 102, 'A'),
    -> (1004, 'Meena', 102, 'A+');
Query OK, 4 rows affected (0.386 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT
    ->     s.student_id,
    ->     s.student_name,
    ->     c.class_name,
    ->     t.teacher_name AS class_teacher,
    ->     s.grade
    -> FROM Student s
    -> JOIN Class c
    ->     ON s.class_id = c.class_id
    -> JOIN Teacher t
    ->     ON c.teacher_id = t.teacher_id;
+------------+--------------+------------+---------------+-------+
| student_id | student_name | class_name | class_teacher | grade |
+------------+--------------+------------+---------------+-------+
|       1001 | Rahul        | Class 10A  | Mr. Kumar     | A     |
|       1002 | Sneha        | Class 10A  | Mr. Kumar     | B     |
|       1003 | Arjun        | Class 10B  | Ms. Priya     | A     |
|       1004 | Meena        | Class 10B  | Ms. Priya     | A+    |
+------------+--------------+------------+---------------+-------+
4 rows in set (0.019 sec)
```

2. **A hospital wants to track patient appointments. Design the relational schema and write**

SQL to find doctors with more than 10 appointments this month.

```
mysql> SELECT
    ->     d.doctor_id,
    ->     d.doctor_name,
    ->     COUNT(*) AS total_appointments
    -> FROM Doctor d
    -> JOIN Appointment a
    -> ON d.doctor_id = a.doctor_id
    -> WHERE MONTH(a.appointment_date) = MONTH(CURDATE())
    -> AND YEAR(a.appointment_date) = YEAR(CURDATE())
    -> GROUP BY d.doctor_id, d.doctor_name
    -> HAVING COUNT(*) > 10;
+-----------+-------------+--------------------+
| doctor_id | doctor_name | total_appointments |
+-----------+-------------+--------------------+
|         1 | Dr. Sharma  |                 13 |
+-----------+-------------+--------------------+
1 row in set (0.014 sec)

mysql> SELECT CURDATE();
+------------+
| CURDATE()  |
+------------+
| 2026-08-04 |
+------------+
1 row in set (0.010 sec)

mysql>
mysql> SELECT * FROM Appointment;
+----------------+------------+-----------+------------------+
| appointment_id | patient_id | doctor_id | appointment_date |
+----------------+------------+-----------+------------------+
|           1001 |        101 |         1 | 2026-08-01       |
|           1002 |        102 |         1 | 2026-08-02       |
|           1003 |        101 |         2 | 2026-08-03       |
|           1004 |        101 |         1 | 2026-08-04       |
|           1005 |        102 |         1 | 2026-08-05       |
|           1006 |        101 |         1 | 2026-08-06       |
|           1007 |        102 |         1 | 2026-08-07       |
|           1008 |        101 |         1 | 2026-08-08       |
|           1009 |        102 |         1 | 2026-08-09       |
|           1010 |        101 |         1 | 2026-08-10       |
|           1011 |        102 |         1 | 2026-08-11       |
|           1012 |        101 |         1 | 2026-08-12       |
|           1013 |        102 |         1 | 2026-08-13       |
|           1014 |        101 |         1 | 2026-08-14       |
+----------------+------------+-----------+------------------+
14 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM Doctor;
+-----------+-------------+----------------+
| doctor_id | doctor_name | specialization |
+-----------+-------------+----------------+
|         1 | Dr. Sharma  | Cardiology     |
|         2 | Dr. Priya   | Neurology      |
+-----------+-------------+----------------+
2 rows in set (0.005 sec)
```

3. **In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.**

```
mysql> INSERT INTO Product VALUES
    -> (1, 'Laptop', 50000),
    -> (2, 'Mouse', 500),
    -> (3, 'Keyboard', 1000),
    -> (4, 'Monitor', 8000),
    -> (5, 'Printer', 12000);
Query OK, 5 rows affected (0.097 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Orders VALUES
    -> (101, '2026-08-01'),
    -> (102, '2026-08-02');
Query OK, 2 rows affected (0.070 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO OrderDetails VALUES
    -> (1, 101, 1, 1),
    -> (2, 101, 2, 2),
    -> (3, 102, 3, 1);
Query OK, 3 rows affected (0.109 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT product_id, product_name
    -> FROM Product
    -> WHERE product_id NOT IN (
    ->     SELECT product_id
    ->     FROM OrderDetails
    -> );
+------------+--------------+
| product_id | product_name |
+------------+--------------+
|          4 | Monitor      |
|          5 | Printer      |
+------------+--------------+
2 rows in set (0.018 sec)

mysql> SELECT p.product_id, p.product_name
    -> FROM Product p
    -> WHERE NOT EXISTS (
    ->     SELECT 1
    ->     FROM OrderDetails od
    ->     WHERE od.product_id = p.product_id
    -> );
+------------+--------------+
| product_id | product_name |
+------------+--------------+
|          4 | Monitor      |
|          5 | Printer      |
+------------+--------------+
2 rows in set (0.014 sec)

mysql>
```

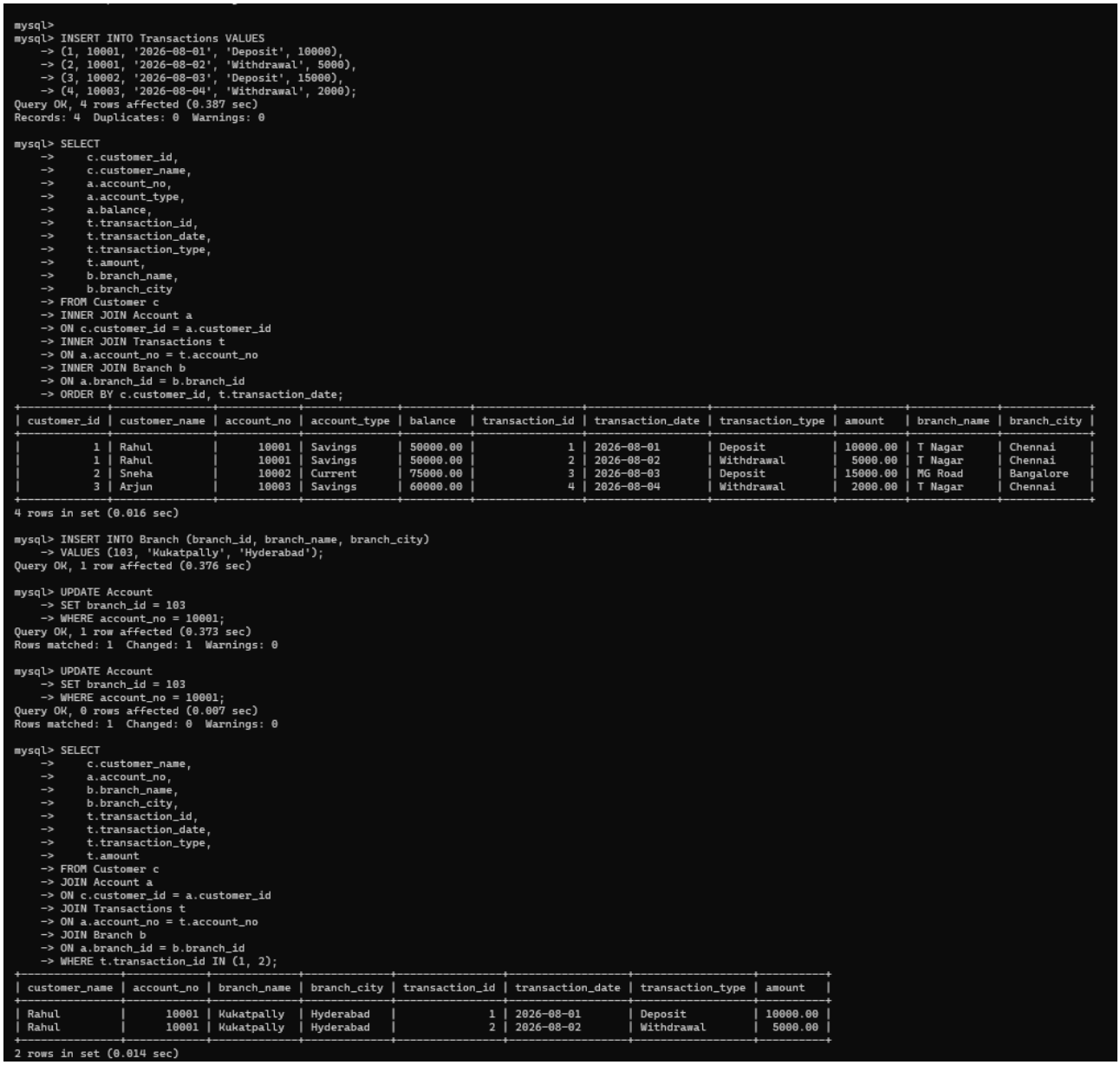
4. **For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.**

```
mysql>
mysql> CREATE TABLE Enrollment (
    ->     student_id INT,
    ->     course_id INT,
    ->     PRIMARY KEY (student_id, course_id),
    ->     FOREIGN KEY (student_id) REFERENCES Student(student_id),
    ->     FOREIGN KEY (course_id) REFERENCES Course(course_id)
    -> );
Query OK, 0 rows affected (0.435 sec)

mysql> INSERT INTO Course VALUES
    -> (1, 'DBMS'),
    -> (2, 'OS'),
    -> (3, 'Java');
Query OK, 3 rows affected (0.402 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Enrollment VALUES
    -> (1001, 1),   -- Rahul -> DBMS
    -> (1001, 2),   -- Rahul -> OS
    -> (1002, 1),   -- Sneha -> DBMS
    -> (1003, 2),   -- Arjun -> OS
    -> (1004, 1),   -- Meena -> DBMS
    -> (1004, 2),   -- Meena -> OS
    -> (1004, 3);   -- Meena -> Java
Query OK, 7 rows affected (0.391 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql> SELECT s.student_id,
    ->        s.student_name
    -> FROM Student s
    -> WHERE s.student_id IN
    -> (
    ->     SELECT e.student_id
    ->     FROM Enrollment e
    ->     JOIN Course c
    ->     ON e.course_id = c.course_id
    ->     WHERE c.course_name = 'DBMS'
    -> )
    -> AND s.student_id IN
    -> (
    ->     SELECT e.student_id
    ->     FROM Enrollment e
    ->     JOIN Course c
    ->     ON e.course_id = c.course_id
    ->     WHERE c.course_name = 'OS'
    -> );
+------------+--------------+
| student_id | student_name |
+------------+--------------+
|       1001 | Rahul        |
|       1004 | Meena        |
+------------+--------------+
2 rows in set (0.031 sec)
```

5. **Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.**

```
Query OK, 0 rows affected (0.419 sec)

mysql> CREATE VIEW BelowDeptAverage AS
    -> SELECT e.emp_id,
    ->        e.emp_name,
    ->        e.department,
    ->        e.salary
    -> FROM Employee e
    -> WHERE e.salary <
    -> (
    ->     SELECT AVG(e2.salary)
    ->     FROM Employee e2
    ->     WHERE e2.department = e.department
    -> );
ERROR 1050 (42S01): Table 'BelowDeptAverage' already exists
mysql> SELECT * FROM BelowDeptAverage;
+--------+----------+------------+----------+
| emp_id | emp_name | department | salary   |
+--------+----------+------------+----------+
|    101 | Rahul    | HR         | 30000.00 |
|    103 | Arjun    | IT         | 50000.00 |
|    105 | Ravi     | IT         | 45000.00 |
|    106 | Priya    | Finance    | 35000.00 |
+--------+----------+------------+----------+
4 rows in set (0.026 sec)

mysql> UPDATE Employee
    -> SET salary = salary * 1.10
    -> WHERE emp_id IN
    -> (
    ->     SELECT emp_id
    ->     FROM BelowDeptAverage
    -> );
Query OK, 4 rows affected, 1 warning (0.463 sec)
Rows matched: 4  Changed: 4  Warnings: 1

mysql> UPDATE Employee
    -> SET salary = salary * 1.10
    -> WHERE emp_id IN
    -> (
    ->     SELECT emp_id
    ->     FROM BelowDeptAverage
    -> );
Query OK, 3 rows affected, 1 warning (0.384 sec)
Rows matched: 3  Changed: 3  Warnings: 1

mysql> SELECT * FROM Employee;
+--------+----------+------------+----------+
| emp_id | emp_name | department | salary   |
+--------+----------+------------+----------+
|    101 | Rahul    | HR         | 36300.00 |
|    102 | Sneha    | HR         | 40000.00 |
|    103 | Arjun    | IT         | 55000.00 |
|    104 | Meena    | IT         | 60000.00 |
|    105 | Ravi     | IT         | 54450.00 |
|    106 | Priya    | Finance    | 42350.00 |
|    107 | Kiran    | Finance    | 45000.00 |
+--------+----------+------------+----------+
7 rows in set (0.005 sec)
```

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.



```
mysql>
mysql> INSERT INTO Transactions VALUES
    -> (1, 10001, '2026-08-01', 'Deposit', 10000),
    -> (2, 10001, '2026-08-02', 'Withdrawal', 5000),
    -> (3, 10002, '2026-08-03', 'Deposit', 15000),
    -> (4, 10003, '2026-08-04', 'Withdrawal', 2000);
Query OK, 4 rows affected (0.387 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT
    ->     c.customer_id,
    ->     c.customer_name,
    ->     a.account_no,
    ->     a.account_type,
    ->     a.balance,
    ->     t.transaction_id,
    ->     t.transaction_date,
    ->     t.transaction_type,
    ->     t.amount,
    ->     b.branch_name,
    ->     b.branch_city
    -> FROM Customer c
    -> INNER JOIN Account a
    -> ON c.customer_id = a.customer_id
    -> INNER JOIN Transactions t
    -> ON a.account_no = t.account_no
    -> INNER JOIN Branch b
    -> ON a.branch_id = b.branch_id
    -> ORDER BY c.customer_id, t.transaction_date;
+-------------+---------------+------------+--------------+----------+----------------+------------------+------------------+----------+-------------+-------------+
| customer_id | customer_name | account_no | account_type | balance  | transaction_id | transaction_date | transaction_type | amount   | branch_name | branch_city |
+-------------+---------------+------------+--------------+----------+----------------+------------------+------------------+----------+-------------+-------------+
|           1 | Rahul         |      10001 | Savings      | 50000.00 |              1 | 2026-08-01       | Deposit          | 10000.00 | T Nagar     | Chennai     |
|           1 | Rahul         |      10001 | Savings      | 50000.00 |              2 | 2026-08-02       | Withdrawal       |  5000.00 | T Nagar     | Chennai     |
|           2 | Sneha         |      10002 | Current      | 75000.00 |              3 | 2026-08-03       | Deposit          | 15000.00 | MG Road     | Bangalore   |
|           3 | Arjun         |      10003 | Savings      | 60000.00 |              4 | 2026-08-04       | Withdrawal       |  2000.00 | T Nagar     | Chennai     |
+-------------+---------------+------------+--------------+----------+----------------+------------------+------------------+----------+-------------+-------------+
4 rows in set (0.016 sec)

mysql> INSERT INTO Branch (branch_id, branch_name, branch_city)
    -> VALUES (103, 'Kukatpally', 'Hyderabad');
Query OK, 1 row affected (0.376 sec)

mysql> UPDATE Account
    -> SET branch_id = 103
    -> WHERE account_no = 10001;
Query OK, 1 row affected (0.373 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Account
    -> SET branch_id = 103
    -> WHERE account_no = 10001;
Query OK, 0 rows affected (0.007 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> SELECT
    ->     c.customer_name,
    ->     a.account_no,
    ->     b.branch_name,
    ->     b.branch_city,
    ->     t.transaction_id,
    ->     t.transaction_date,
    ->     t.transaction_type,
    ->     t.amount
    -> FROM Customer c
    -> JOIN Account a
    -> ON c.customer_id = a.customer_id
    -> JOIN Transactions t
    -> ON a.account_no = t.account_no
    -> JOIN Branch b
    -> ON a.branch_id = b.branch_id
    -> WHERE t.transaction_id IN (1, 2);
+---------------+------------+-------------+-------------+----------------+------------------+------------------+----------+
| customer_name | account_no | branch_name | branch_city | transaction_id | transaction_date | transaction_type | amount   |
+---------------+------------+-------------+-------------+----------------+------------------+------------------+----------+
| Rahul         |      10001 | Kukatpally  | Hyderabad   |              1 | 2026-08-01       | Deposit          | 10000.00 |
| Rahul         |      10001 | Kukatpally  | Hyderabad   |              2 | 2026-08-02       | Withdrawal       |  5000.00 |
+---------------+------------+-------------+-------------+----------------+------------------+------------------+----------+
2 rows in set (0.014 sec)
```

7. A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

```
mysql> CREATE TABLE Sales (
    ->     sale_id INT PRIMARY KEY,
    ->     product_name VARCHAR(50),
    ->     category VARCHAR(50),
    ->     sale_date DATE,
    ->     quantity INT,
    ->     amount DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.501 sec)

mysql> INSERT INTO Sales VALUES
    -> (1, 'Laptop', 'Electronics', '2026-08-04', 1, 50000),
    -> (2, 'Mouse', 'Electronics', '2026-08-04', 5, 2500),
    -> (3, 'Keyboard', 'Electronics', '2026-08-04', 4, 4000),
    -> (4, 'Rice Bag', 'Groceries', '2026-08-04', 10, 8000),
    -> (5, 'Oil', 'Groceries', '2026-08-04', 5, 3000),
    -> (6, 'Shampoo', 'Personal Care', '2026-08-04', 10, 4500),
    -> (7, 'Soap', 'Personal Care', '2026-08-04', 20, 2500);
Query OK, 7 rows affected (0.387 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql> SELECT
    ->     category,
    ->     SUM(amount) AS total_sales
    -> FROM Sales
    -> WHERE sale_date = '2026-08-04'
    -> GROUP BY category
    -> HAVING SUM(amount) > 10000;
+-------------+-------------+
| category    | total_sales |
+-------------+-------------+
| Electronics |    56500.00 |
| Groceries   |    11000.00 |
+-------------+-------------+
2 rows in set (0.011 sec)
```

8. For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

$$
\begin{array}{c}
\{ \\
m.member\_name \mid \\
Member(m) \text{ AND} \\
\text{FOR ALL } g \text{ (} \\
Book(g) \rightarrow \\
\text{EXISTS } b \text{ EXISTS } br \text{ (} \\
Book(b) \text{ AND} \\
Borrow(br) \text{ AND} \\
b.book\_id = br.book\_id \text{ AND} \\
br.member\_id = m.member\_id \text{ AND} \\
b.genre = g.genre \\
) \\
) \\
\}
\end{array}
$$

Explanation

- Member(m) selects each library member.
- FOR ALL g checks **every genre** available in the Book relation.
- EXISTS b ensures there is a book in that genre.
- EXISTS br verifies that the member has borrowed a book belonging to that genre.
- Members satisfying this condition have borrowed books from **all genres** available in the library.

9.Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL

```
mysql> CREATE TABLE Supplier (
    ->     supplier_id INT PRIMARY KEY,
    ->     supplier_name VARCHAR(50),
    ->     city VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'supplier' already exists
mysql>
mysql> CREATE TABLE Product (
    ->     product_id INT PRIMARY KEY,
    ->     product_name VARCHAR(100),
    ->     supplier_id INT,
    ->     quantity INT CHECK (quantity >= 0),
    ->     price DECIMAL(10,2) CHECK (price > 0),
    ->     FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
    -> );
ERROR 1050 (42S01): Table 'product' already exists
mysql> INSERT INTO Supplier VALUES
    -> (1,'ABC Suppliers','Hyderabad'),
    -> (2,'XYZ Traders','Chennai');
Query OK, 2 rows affected (0.408 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Product VALUES
    -> (101,'Laptop',1,20,55000),
    -> (102,'Mouse',2,100,500);
Query OK, 2 rows affected (0.222 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT TABLE_NAME
    -> FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
    -> WHERE REFERENCED_TABLE_NAME = 'Product'
    -> AND TABLE_SCHEMA = DATABASE();
+--------------+
| TABLE_NAME   |
+--------------+
| orderdetails |
+--------------+
1 row in set (0.018 sec)

mysql> Select * From supplier
    -> ;
+-------------+---------------+-----------+
| supplier_id | supplier_name | city      |
+-------------+---------------+-----------+
|           1 | ABC Suppliers | Hyderabad |
|           2 | XYZ Traders   | Chennai   |
+-------------+---------------+-----------+
2 rows in set (0.005 sec)

mysql> Select * from Product;
+------------+--------------+-------------+----------+----------+
| product_id | product_name | supplier_id | quantity | price    |
+------------+--------------+-------------+----------+----------+
|        101 | Laptop       |           1 |       20 | 55000.00 |
|        102 | Mouse        |           2 |      100 |   500.00 |
+------------+--------------+-------------+----------+----------+
2 rows in set (0.005 sec)
```

10. Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

```
mysql> SELECT * FROM Attendance;
+---------------+--------+-----------------+----------+
| attendance_id | emp_id | attendance_date | status   |
+---------------+--------+-----------------+----------+
|             1 |    101 | 2026-08-01      | Present  |
|             2 |    101 | 2026-08-02      | Present  |
|             3 |    102 | 2026-08-01      | Absent   |
|             4 |    102 | 2026-08-02      | Present  |
|             5 |    103 | 2026-08-01      | Present  |
|             6 |    104 | 2026-08-01      | Present  |
|             7 |    105 | 2026-08-01      | Absent   |
|             8 |    106 | 2026-08-01      | Present  |
|             9 |    107 | 2026-08-01      | Present  |
+---------------+--------+-----------------+----------+
9 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM Performance;
+----------------+--------+--------------+-------------------+
| performance_id | emp_id | review_month | performance_score |
+----------------+--------+--------------+-------------------+
|              1 |    101 | August       |                90 |
|              2 |    102 | August       |                85 |
|              3 |    103 | August       |                88 |
|              4 |    104 | August       |                92 |
|              5 |    105 | August       |                80 |
|              6 |    106 | August       |                87 |
|              7 |    107 | August       |                89 |
+----------------+--------+--------------+-------------------+
7 rows in set (0.005 sec)

mysql>
mysql> SELECT * FROM Monthly_Attendance;
+--------+------------+---------------+
| emp_id | total_days | present_days  |
+--------+------------+---------------+
|    101 |          2 |             2 |
|    102 |          2 |             1 |
|    103 |          1 |             1 |
|    104 |          1 |             1 |
|    105 |          1 |             0 |
|    106 |          1 |             1 |
|    107 |          1 |             1 |
+--------+------------+---------------+
7 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM Performance_Summary;
+--------+-----------------+
| emp_id | average_score   |
+--------+-----------------+
|    101 |         90.0000 |
|    102 |         85.0000 |
|    103 |         88.0000 |
|    104 |         92.0000 |
|    105 |         80.0000 |
|    106 |         87.0000 |
|    107 |         89.0000 |
+--------+-----------------+
7 rows in set (0.010 sec)
```